



Fichiers



master ▾



projetWelab /

Parcourir

Filtrer

Source

Description

Dernière modification



ProjetWelab-main



README.md

Ajout du readme

04 mai 2026



README.md

Projet Informatique – Gestion des Plannings

WeLab Cosmetic



Membres du Groupe

- **Moussoni Joan** – joan.moussoni@etu.univ-orleans.fr
- **Sambou Aicha** – aicha.sambou@etu.univ-orleans.fr
- **Diallo Fanta** – fanta.diallo@etu.univ-orleans.fr

Encadrant : Mr Khalil Djelloul

Description du Projet

Ce projet a pour objectif de développer une application web de **gestion des plannings et des réservations** pour WeLab Cosmetic.

Architecture du Projet

Le projet est structuré en deux parties :

◆ Frontend

- Framework : **Angular**
- Utilisation du Router Angular
- Composants standalone
- Interface utilisateur moderne et responsive

◆ Backend

- Framework : **Symfony**
- API REST
- Gestion des données (plannings, utilisateurs, réservations ect...)

Début du Projet – Création Angular

Création du projet Angular

```
ng new welab-angular
```

Lancer le serveur de développement

```
ng serve --host 0.0.0.0 --port 4200
```

Pages Intégrées

Home

- Page principale
- nav bar lien vers les pages (connexion; inscription; a propos; contact)
- Bouton de prise de rendez-vous qui mene vers la page de connexion

Accueil

- Page d'accueil utilisateur connecté
- Interface cohérente avec Home
- Navigation via Angular Router

i About (À propos)

- Présentation de l'entreprise WeLab Cosmetic
- Description des activités
- Bouton de redirection vers Contact

Contact

- Informations officielles de l'entreprise
- Adresse
- Téléphone
- Email

commande pour les pages

```
ng generate component pages/home
ng generate component pages/accueil
ng generate component pages/about
ng generate component pages/contact
```

Pages Intégrées (fanta)

Connexion

- Formulaire de connexion utilisateur
- Champs email + mot de passe

Inscription

- Formulaire d'inscription utilisateur
- Création d'un nouveau compte
- Validation des champs

Notifications

- Affichage des notifications utilisateur

Documents

- Liste des documents disponibles
- Consultation des documents

commande pour générer les pages

```
bash ng generate component pages/connexion ng generate component pages/inscription ng generate component pages/notifications ng generate component pages/documents
```

Création d'un dossier assets pour que la vague en bas de page de tout les pages et pour que les images associées aux autres pages y soit stocker.

- wave-green.png
- logo_facebook.png
- logo_google.png

Création d'un dossier services

- auth.service.ts est le Service d'authentification Angular. Il sert d'intermédiaire entre le frontend Angular et le backend Symfony pour gérer la connexion.

Creation du Projet Symfony pour le backend

- commande : `symfony new WelabSymfony --weapp`

Creation des entités (Fanta)

- Creation du controleur: `symfony console make:controller WelabController`
- Installation de Doctrine et du maker bundle
- Commande Doctrine: `composer require symfony/orm-pack`
- Commande du maker bundle `composer require --dev symfony/maker-bundle`
- Creation des entity:
- Utilisateur.php, Prestation.php, MaterielleMobile.php, Reservation.php, ProjetTutore.php,Projet.php, PreuveConcept.php, Laboratoire.php, ChallengeEtudiant.php, Atelier.php, Administrateur.php, Document.php, GestionLabo.php, GestionStage.php, Notification.php, Possede.php, Stage.php,
- Commande pour la création d'une entité: `symfony console make:entity Nomtable`

les migrations : (Aïcha)

J'ai mis en place la structure de la base de données avec **Doctrine Migrations** dans Symfony.

Commandes utilisées

Créer une migration :

```
symfony console make:migration
```

Permet de générer un fichier de migration à partir des entités. Exécuter les migrations :

```
symfony doctrine:migrations:migrate
```

Permet d'appliquer les modifications dans la base de données (création des tables, relations, etc.). Grâce à ces commandes, la base de données est générée automatiquement à partir des entités du projet.

Travail à continuer

Ma collègue devra :

- Installer **Faker** et **Doctrine Fixtures**
- Créer les **fixtures** pour générer des données de test
- Charger les données dans la base ect

Fonctionnalités réalisées par Jaon

- Création d'une barre de navigation avec accès aux différentes pages (Accueil, Projets, Contact, À propos).
- Création d'une page Projets avec plusieurs catégories :
- Preuve de concept
 - Atelier
 - Challenge étudiant
 - Projet tutoré
 - Autre
- Mise en place de la navigation entre les pages avec Angular Router.
- Création d'un formulaire de soumission de projet comprenant :
 - Nom du projet
 - Date de début
 - Date de fin
 - Descriptif
 - Ajout d'un fichier
 - Ajout d'un bouton de soumission centré.
- Mise en place du style CSS de l'interface (navbar, boutons, formulaire, footer).
- Intégration d'un footer avec vague graphique.

Objectif

- Permettre aux utilisateurs de sélectionner un type de projet et de soumettre les informations nécessaires via un formulaire simple et accessible.

Creation de la bdd en local (Fanta)

- nous avons décidé de crée la bdd en local pour nous simplifier la gestion des données
- creation de la bdd avec la commande:
 - `symfony console make:migration`

- symfony console doctrine:migrations:migrate

Creation des fixtures : (Fanta)

Les fixtures permettent de remplir automatiquement la base avec des données de test pour développer et vérifier l'application rapidement. Le Faker est une bibliothèque qui génère automatiquement des données réalistes (noms, emails, dates...) pour rendre les fixtures variées et crédibles.

- installation fixtures commande: composer require orm-fixtures --dev
- installation faker commande: composer require fakerphp/faker --dev
- la commande pour créer les fixtures et le dossier fixture si il n'est pas créer: symfony console make:fixture nomDeLaFixture
- les fixtures créer sont : - AdministrateurFixtures.php - AppFixtures.php - AtelierFixtures.php - ChallengeEtudiantFixtures.php - DocumentFixtures.php - GestionLaboFixtures.php - GestionStageFixtures.php - LaboratoireFixtures.php - MaterielleMobileFixtures.php - NotificationFixtures.php - ProjetFixtures.php - PossedeFixtures.php - StageFixtures.php - UtilisateurFixtures.php - ReservationFixtures.php
- La commande pour charger les fixtures dans la bdd : symfony console doctrine:fixtures:load

connexion (fanta)

- connexion des différents rôle: - admin prendre l'email dans la bdd et le mot de passe est 'admin123'. - utilisateur prendre l'email dans la bdd et le mot de passe est 'password123'.
- Commande CORS qui est une configuration backend où Symfony autorise Angular à lui envoyer des requêtes : symfony composer require nelmio/cors-bundle
- Configuration du fichier config/packages/nelmio_cors.yaml pour autoriser Angular à envoyer des requêtes à Symfony en mettant le port externe d'Angular dans allow_origin.
- Création du contrôleur Symfony qui gère la connexion : - la commande: symfony console make:controller AuthController
- Configuration du fichier config/routes.yaml pour charger les contrôleurs.
- Le mot de passe est hashé avec MD5 (32 caractères) pour respecter la contrainte length: 32 de la base de données. L'URL de l'API Symfony dans le service Angular src/services/auth.service.ts doit correspondre au port externe de Symfony (8010).
- Activation de HttpClient dans src/app/app.config.ts sans quoi les appels HTTP ne fonctionnaient pas.
- En cas de succès, le token est stocké dans le localStorage et l'utilisateur est redirigé vers la page d'accueil dans le cas contraire un message d'erreur est affiché sur la page.

Creation des pages admin(Fanta)

- Les pages générées sont:
 - dashboard,
 - reservations-admin,
 - stages-admin,
 - prestations-admin,
 - materiaux-admin,
 - laboratoires-admin,
 - messages-admin
- Les pages n'ont pas encore été styliser je ne compte m'occuper que du dashboard, de la barre de navigation de nos utilisateurs et des visiteurs.

Fonctionnalités ajoutées (Aicha)

Page Planning

Une page planning a été réalisée côté frontend avec Angular.

Cette page permet :

- de sélectionner un **projet validé** dans une liste déroulante
- d'afficher un **calendrier**
- de choisir une **date disponible**
- d'afficher les **créneaux horaires disponibles**
- de choisir un **laboratoire**
- d'appuyer sur le bouton **Soumettre**

Le planning a été conçu d'abord côté frontend pour mettre en place l'interface et la logique d'affichage.

Fonctionnalité Inscription

La page inscription a été reliée au backend Symfony.

Elle permet :

- de saisir les informations utilisateur :
 - nom
 - prénom

- email
- téléphone
- mot de passe
- d'envoyer les données au backend
- d'enregistrer un nouvel utilisateur dans la base de données
- de permettre ensuite à cet utilisateur de se connecter avec ses identifiants
- Liaison Angular / Symfony Pour ces fonctionnalités :
- Angular envoie les données avec `HttpClient`
- Symfony reçoit les données via des routes API
- les informations sont enregistrées dans la base de données

État actuel

À ce stade :

- la page **Planning** est en cours de finalisation côté frontend
- la fonctionnalité **Inscription** est mise en place avec le backend
- l'utilisateur inscrit peut ensuite utiliser ses identifiants pour se connecter

Profil Utilisateur

Objectif

Afficher les informations de l'utilisateur connecté et gérer ses réservations.

- Affichage :
 - Nom
 - Prénom
 - Email
 - Téléphone

◆ Réservations

- réservations envoyées
- réservations en attente

◆ Projets

- Projets validés →
- Projets en attente →

Annulation réservation

Backend

DELETE /api/reservation/annuler/{id}

Front

- Confirmation utilisateur
- Suppression en base
- Mise à jour immédiate de l'affichage
- Message de succès

Accès rapide

- Documents → `/document`
- Notifications → `/notifications`
- Déconnexion

1. Notification admin

- Lors d'une annulation
- Création d'une notification en base

2. Déplacement réservation (réel)

- redirection vers planning avec la réservation
- Modifier en base

Fonctionnnalités (Fanta)

Liste contact des utilisateurs pour les admins

- L'admin peut consulter la liste de tous les utilisateurs (nom, prénom, email, téléphone, rôle) depuis l'onglet Contact du dashboard.
- création de la page liste-contact pour les admins : `ng generate component pages/admin/liste-contact`
- Création du contrôleur Symfony qui expose l'endpoint : `symfony console make:controller AdminController`

Gestion des prestations

- L'admin peut consulter la liste de toutes les prestations, les modifier, supprimer et ajouter de nouvelles prestations.
- modification des pages prestations-admin et du contrôleur `AdminController`.
- Une barre verte s'affiche en haut de la page en cas de succès et une barre rouge en cas d'erreur. Les messages disparaissent automatiquement après 3 secondes.

Gestion des projets et des réservations dans le dashboard

- Affichage des des réservations et des projets contenue dans la bdd
- creation d'une relation utilisateur et projet car on ne pouvait pas identifier quel utilisateur est relié à un projet et une reservation.
- Gestion des ajouts, des suppressions et des annulations des projet et réservations
- Les réservations passées ne sont pas affichées dans réservation validée
- Chaque réservation ou projet possède :
 - une mini liste affichant les projets et réservations où on peut sélectionner un élément précis, cet élément est directement affiché dans la carte.
 - Cela évite l'affichage aléatoire et permet une gestion ciblée.
 - une vue détaillée accessible via bouton `☰` → Détails
- Depuis la vue détaillée, l'admin peut :
 - écrire un message et l'envoyer à l'utilisateur
- Routes principales :
 - GET `/api/admin/reservations/attente`
 - GET `/api/admin/reservations/validees`
 - GET `/api/admin/projets/attente`
 - GET `/api/admin/projets/valides`
 - GET `/api/admin/reservations/{id}/details`
 - GET `/api/admin/projets/{id}/details`
 - POST `/api/admin/reservations/{id}/message`
 - POST `/api/admin/projets/{id}/message`

Gestion des Laboratoires

- Affichage des Laboratoires.
- Possibilité d'ajouter, de supprimer et de modifier un laboratoire.
- Gérer la disponibilité : - Disponible - Occupé - Maintenance
- Création du contrôleur `LaboratoireController.php`
- Les methodes créer pour ces fonctionnalités sont: - Lister les laboratoires: GET `/api/laboratoires` - Ajouter un laboratoire: POST `/api/laboratoires` - Modifier un laboratoire: PUT `/api/laboratoires/{id}` - Supprimer un laboratoire: DELETE `/api/laboratoires/{id}`

Gestion de l'affichage des réservations pour l'admin

- L'admin voit toutes les réservations (par défaut celles passées ne sont pas affichées), il aura des filtres d'affichage par statut: - EN_ATTENTE - VALIDEE - REFUSEE
- Bouton pour afficher toutes les réservations (historique inclus)
- Ajout de statut dans table projet pour facilité l'affichage et la gestion des projets pour les admins.

Profil admin

- Creation des pages profil-admin commande= `ng generate component pages/admin/profil-admin`
- creation d'une methode réinitialisation de mot de passe :PUT `/api/admin/profil/{id}/reset-password`
- affichage du mail de l'admin et de son id : GET `/api/admin/profil/{id}`
- Sécurité via vérification de l'ancien mot de passe
- Option Afficher / Masquer les mots de passe
- Les mots de passe sont hashés en MD5
- Le front envoie les mots de passe en clair → le backend les hash

Fonctionnnalités (aicha)

Creation des pages:

Mes Réservations

- Voir toutes les réservations

- Filtrer par statut
- Détails / Annuler / Déplacer
- Déplacement → planning prérempli

Mes Projets

- Voir tous les projets
- Filtrer par statut
- Détails
- Prendre rendez-vous (si validé)

modification de profil

- Affiche 1 seule réservation (récente)
- Affiche 1 seul projet (récent)
- Boutons : Annuler / Déplacer / RDV
- Sert de raccourci vers pages complètes

quelque modifs sur l'inscription

Sur mon cote de l'inscription , l'utilisateur avait deja le role utilisateur des son inscription, j'ai juste appoter quelque modifs pour mieux le prendre en compte (methode register dans authController)

Gestion des documents

Gérer les documents liés aux projets : affichage, ajout et téléchargement.

Fonctionnalités

- Affichage des documents d'un projet (`/document/:id`)
- Upload de fichiers (FormData)
- Téléchargement et visualisation des documents

API (Symfony)

- `GET /api/projets/{id}/documents`
- `POST /api/projets/{id}/documents`
- `GET /api/documents/{id}/download`

Stockage

Fichiers enregistrés dans :

`public/uploads/documents/`

Gestion des réservations de matériel

Fonctionnement général

L'utilisateur peut consulter la liste de ses réservations depuis son espace personnel.

- Chaque réservation possède un statut (`EN_ATTENTE` , `VALIDEE` , `REFUSEE`)
- Lorsqu'une réservation est **VALIDEE**, une action supplémentaire devient disponible

Ajout de matériel

Si une réservation est au statut **VALIDEE**, un bouton "**Ajouter matériel**" apparaît.

Étapes côté utilisateur

1. L'utilisateur clique sur "**Ajouter matériel**"
2. Une fenêtre (modal) s'ouvre avec la liste des matériels disponibles
3. L'utilisateur choisit un matériel à ajouter

Traitement côté backend

Lors de l'ajout d'un matériel, plusieurs vérifications sont effectuées :

-  La réservation doit être **VALIDEE**
-  Le matériel doit exister
-  Le matériel doit être disponible (`nbDispo > 0`)
-  Le matériel ne doit pas déjà être lié à cette réservation

Affichage des résultats

Après ajout :

- Le matériel apparaît dans les **détails de la réservation**
- L'utilisateur peut voir tous les matériels associés à chaque réservation

Planning & Réservation — Welab

Module permettant de réserver un laboratoire pour un projet validé via Angular + API Symfony.

Frontend (Angular)

- Projets validés de l'utilisateur
- Liste des laboratoires (3 labos métier)
- Parcours guidé : **Projet** → **Labo** → **Date** → **Horaire**
- Calendrier dynamique :
 - dates futures uniquement
 - jours réellement disponibles
- Chargement des créneaux selon labo + date
- Messages d'erreur + confirmation visuelle

✓ Backend (Symfony)

Endpoints

- GET /api/projets-valides/{idUtilisateur}
- GET /api/laboratoires
- GET /api/planning/{idLabo}/{date}
- GET /api/planning/disponibilites-jours/{idLabo}/{annee}/{mois}
- POST /api/reservations

Logique métier

- Créneaux fixes (08-10, 10-12, 14-16, 16-18)
- Jours dispo = au moins 1 créneau libre + non passé
- Vérifications :
 - projet valide
 - date future
 - créneau non déjà réservé
- Statuts : `EN_ATTENTE`

Gestions des Projets

Création de projet (Angular + Symfony)

- Objectif : Permettre à un utilisateur de créer un projet et de l'enregistrer en base de données.
- Frontend : création d'un formulaire Angular avec récupération des données utilisateur.
- Gestion du ngModel, validation des champs et envoi HTTP POST vers l'API.
- Backend : création d'une route /api/ajout-challenge.
- Traitement : récupération JSON, vérification des données, création entité Projet, association utilisateur et prestation.

Système de notifications

Objectif : informer les admins des actions importantes. Création d'une entité Notifications avec message, statut, utilisateur et administrateur. Implémentation des routes :

- GET : récupérer les notifications
- PUT : marquer comme lue
- DELETE : supprimer Création automatique d'une notification lors de l'ajout d'un projet. Gestion complète du cycle de vie d'une notification.

Affichage des stages

Création d'une route Symfony pour récupérer tous les stages. Transformation des données côté backend (format date, champs utiles). Côté Angular : appel API avec HttpClient. Stockage dans un tableau et affichage dynamique avec *ngFor. Affichage des informations principales : sujet, dates, rémunération, formation.

Gestion des documents

Création d'une API permettant de récupérer tous les documents. Transformation des données pour inclure relations : utilisateur, projet, stage. Structuration des données JSON pour le frontend. Côté Angular : affichage sous forme de tableau avec nom, type, utilisateur, projet et lien.

Visualisation des documents

Création d'une route permettant de voir un document via son ID. Utilisation de BinaryFileResponse pour retourner le fichier.

Vérification de l'existence du fichier avant affichage. Correction d'un bug important : duplication du chemin /uploads. Mise en place d'un accès sécurisé aux fichiers.

Problèmes rencontrés et solutions

Erreur 404 sur les documents → corrigée en nettoyant les chemins. Problèmes API Angular (status 0) → liés au port ou backend non lancé. Gestion des erreurs avec messages utilisateur côté frontend. Debug avec logs Symfony et requêtes SQL.

Compétences mises en œuvre

- Angular : formulaires, services HTTP, affichage dynamique
- Symfony : création d'API REST, gestion des entités Doctrine
- Base de données : relations entre entités
- Debug : analyse erreurs backend/frontend
- Architecture : communication Front ↔ Back

Cette gestion des dépôts Git est procurée par Atlassian Bitbucket

[Atlassian Bitbucket v5.11.1](#) · [Documentation](#) · [Contacter le service d'assistance](#) · [Demander une fonctionnalité](#) · [À propos de](#) · [Contacter Atlassian](#)